

Experiment 'p_6_2_h_is_sorted_seq' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_2_h_is_sorted_seq

Natural language statement: Write a method to return true if the given array is currently sorted in increasing order.

Method signature: p_6_2_h_is_sorted_seq(inputs: seq<int>) returns (is_sorted: bool)

Ensures

- `is_sorted == (forall i :: 0 <= i < |inputs| - 1 ==> inputs[i] <= inputs[i + 1])`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to return true if the given array is currently sorted in increasing order.

The signature should be:

`p_6_2_h_is_sorted_seq(inputs: seq<int>) returns (is_sorted: bool)`

The method should respect the following contract:

`ensures is_sorted == (forall i :: 0 <= i < |inputs| - 1 ==> inputs[i] <= inputs[i + 1])`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn1D25ouKhFwi5VHK87x7v5Ia3WEr

Model: gpt-5.5-2026-04-23

Created at: 1780575092

Finish reason: stop

Usage: 314 tokens in, and 2933 tokens out

```
{
  if |inputs| < 2 {
    is_sorted := true;
    return;
  }

  var i := 0;
  while i < |inputs| - 1
    invariant 0 <= i <= |inputs| - 1
    invariant forall k :: 0 <= k < i ==> inputs[k] <= inputs[k + 1]
    decreases |inputs| - 1 - i
  {
    if inputs[i] > inputs[i + 1] {
      is_sorted := false;
      assert 0 <= i < |inputs| - 1;
      assert !(inputs[i] <= inputs[i + 1]);
      assert !(forall k :: 0 <= k < |inputs| - 1 ==> inputs[k] <= inputs[k + 1]);
      return;
    }

    forall k | 0 <= k < i + 1
```

```

    ensures inputs[k] <= inputs[k + 1]
  {
    if k == i {
      assert inputs[k] <= inputs[k + 1];
    } else {
      assert k < i;
      assert inputs[k] <= inputs[k + 1];
    }
  }

  i := i + 1;
}

assert i == |inputs| - 1;
assert forall k :: 0 <= k < |inputs| - 1 ==> inputs[k] <= inputs[k + 1];
is_sorted := true;
}

```

Final Program

```

method p_6_2_h_is_sorted_seq(inputs: seq<int>) returns (is_sorted: bool)
  ensures is_sorted == (forall i :: 0 <= i < |inputs| - 1 ==> inputs[i] <= inputs[i
    ↪ + 1])
{
  if |inputs| < 2 {
    is_sorted := true;
    return;
  }

  var i := 0;
  while i < |inputs| - 1
    invariant 0 <= i <= |inputs| - 1
    invariant forall k :: 0 <= k < i ==> inputs[k] <= inputs[k + 1]
    decreases |inputs| - 1 - i
  {
    if inputs[i] > inputs[i + 1] {
      is_sorted := false;
      assert 0 <= i < |inputs| - 1;
      assert !(inputs[i] <= inputs[i + 1]);
      assert !(forall k :: 0 <= k < |inputs| - 1 ==> inputs[k] <= inputs[k + 1]);
      return;
    }

    forall k | 0 <= k < i + 1
      ensures inputs[k] <= inputs[k + 1]
    {
      if k == i {
        assert inputs[k] <= inputs[k + 1];
      } else {
        assert k < i;

```

```
        assert inputs[k] <= inputs[k + 1];
    }
}

    i := i + 1;
}

assert i == |inputs| - 1;
assert forall k :: 0 <= k < |inputs| - 1 ==> inputs[k] <= inputs[k + 1];
is_sorted := true;
}
```

Total Token Usage

Input tokens: 314

Output tokens: 2933

Reasoning tokens: 2560

Sum of 'total tokens': 3247

Experiment Timings

Overall Experiment started at 1780575091602, ended at 1780575196805, lasting 105203ms (105.20 seconds)

Iteration #1 started at 1780575091604, ended at 1780575196805, lasting 105201ms (105.20 seconds)